

第七次编程实践课

软件工程与计算 I · 2026 · 南京大学

| Week 7 · 输入输出与文件操作 · System Prompt 上下文工程

今日安排

知识讲解

Java 输入输出与文件操作

Scanner · BufferedReader/Writer · Files · try-with-resources

AI实践 上下文工程 — System Prompt

四要素框架 · 安全边界 · 迭代优化

大作业 05-Env 答案解析 · 06-Control 布置

环境与绑定参考答案点评 · 控制流任务讲解

当堂练习

练习 Lab07 · 共 2 题，独立完成

- 练习 1: 按行读取文件 + 词频统计
- 练习 2: 文件复制与追加写入

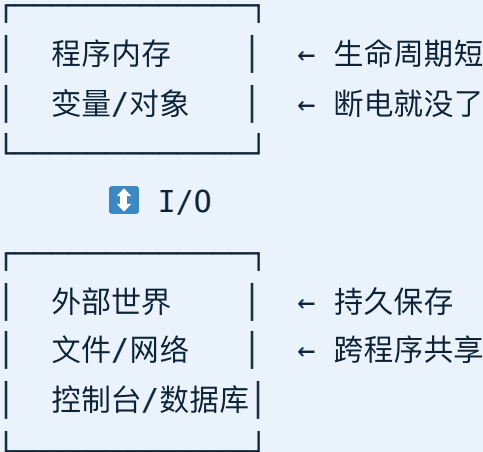
目标: 掌握 `try-with-resources` 资源管理铁律, 理解文本 I/O 的三种主流写法

PART 1 · Java 知识讲解

输入输出与文件操作

为什么需要 I/O?

程序与外部世界的桥梁



大作业中无处不在

场景	用到什么
REPL 读取用户输入	<code>BufferedReader</code>
读取源代码文件	<code>Files.readAllLines</code>
输出错误消息	<code>System.err</code>
日志写入	<code>BufferedWriter</code>
测试数据加载	<code>Scanner</code>

没有 I/O，程序就是一个孤岛——无法与用户、文件、其他程序通信。

Java I/O 两大分支：字节流 vs 字符流

类别	处理对象	基类	典型实现
字节流	二进制数据（图片、视频、任意文件）	InputStream / OutputStream	FileInputStream · FileOutputStream
字符流	文本数据（按字符 / 按行）	Reader / Writer	FileReader · FileWriter · BufferedReader

何时用哪个？

处理文本（.txt, .java, .csv, .md） → 字符流
处理二进制（.jpg, .zip, .class, .db） → 字节流

记忆口诀：“文”字符流， “二”字节流——看到 Reader / Writer 就是文本，看到 InputStream / OutputStream 就是字节。

标准输入输出 — Scanner 与 System.out

控制台输入 Scanner

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
String name = sc.nextLine(); // 读一行
int age = sc.nextInt();
double score = sc.nextDouble();

sc.close(); // 记得关闭!
```

方法	作用
nextLine()	读一整行（含空格）
next()	读一个单词
nextInt()	读一个整数
hasNext()	是否还有输入

控制台输出

```
System.out.println("Hello"); // 换行
System.out.print("no newline"); // 不换行

// 格式化输出
System.out.printf("%d+%d=%d\n", 1, 2, 3);
// %d整数 %s字符串 %.2f保留2位 %n换行

System.err.println("Error!"); // 错误流
```

大作业 REPL 中

```
BufferedReader br = new BufferedReader(
    new InputStreamReader(System.in));
String line = br.readLine();
```

File 类 — 文件元信息操作

```
import java.io.File;

File file = new File("data.txt");

file.exists();           // 是否存在 → boolean
file.isFile();           // 是否为普通文件
file.isDirectory();      // 是否为目录
file.length();           // 文件大小（字节）
file.getName();          // 文件名 "data.txt"
file.getAbsolutePath();  // 绝对路径
file.lastModified();     // 最后修改时间
file.createNewFile();    // 创建新文件
file.delete();           // 删除
file.mkdirs();           // 创建多级目录
file.listFiles();        // 列出子文件（目录时）
```

File 只操作"元信息"（名字、大小、路径）——要读写内容，需要用 `Reader` / `Writer`。

文件读取 — 方式 1: BufferedReader (推荐)

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {    // 按行读, 读到 null 为止
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

为什么要"Buffered"?

	普通 <code>FileReader</code>	<code>BufferedReader</code>
读取单位	每次 1 个字符 → 频繁磁盘 I/O	一次读一块到内存
性能	慢	快 10~100 倍
支持 <code>readLine()</code>	否	是

铁律：读文本文件永远用 `BufferedReader` 包装 `FileReader`。

文件读取 — 方式 2: Files (Java 7+)

一次性读所有行

```
import java.nio.file.*;

List<String> lines = Files.readAllLines(
    Paths.get("data.txt"));

for (String line : lines) {
    System.out.println(line);
}
```

Java 11+ 读整个文件

```
String content = Files.readString(
    Paths.get("data.txt"));
```

流式读取 (大文件推荐)

```
try (Stream<String> stream =
    Files.lines(Paths.get("big.txt"))) {
    stream.filter(l -> l.contains("ERROR"))
        .forEach(System.out::println);
}
```

三种方式对比

方式	适用场景
<code>readAllLines</code>	小文件，需索引
<code>readString</code>	整体处理字符串
<code>lines</code>	大文件流式

`readAllLines` 最常用。

文件写入 — 三种方式

方式 1: BufferedWriter

```
try (BufferedWriter bw = new BufferedWriter(  
    new FileWriter("out.txt"))) {  
    bw.write("Hello");  
    bw.newLine();  
    bw.write("World");  
}
```

方式 2: PrintWriter

```
try (PrintWriter pw =  
    new PrintWriter("out.txt")) {  
    pw.println("Hello");  
    pw.printf("%d+%d=%d\n", 1, 2, 3);  
}
```

方式 3: Files

```
// 写多行  
Files.write(Paths.get("out.txt"),  
    Arrays.asList("Line1", "Line2"));  
  
// Java 11+  
Files.writeString(Paths.get("out.txt"),  
    "Hello World");
```

追加写入（不覆盖）

```
// 第2参数 true = 追加  
new FileWriter("log.txt", true);
```

默认写入会覆盖原文件！

try-with-resources — 资源管理铁律

✗ 传统写法 (易忘记 close)

```
BufferedReader br = null;
try {
    br = new BufferedReader(
        new FileReader("x.txt"));
    // 读取代码
} finally {
    if (br != null) {
        try { br.close(); }
        catch (IOException e) { }
    }
}
```

忘记 `close()` → 资源泄露!

✓ try-with-resources (Java 7+)

```
try (BufferedReader br = new BufferedReader(
    new FileReader("x.txt"))) {
    // 自动 close, 无需 finally
}
```

多资源同时声明

```
try (BufferedReader br = ...;
    BufferedWriter bw = ...) {
    // 两个都会自动 close
}
```

铁律: 任何实现 `AutoCloseable` 的资源都要用 `try-with-resources`。

I/O 异常处理

```
try {
    BufferedReader br = new BufferedReader(new FileReader("x.txt"));
    // ...
} catch (FileNotFoundException e) {    // ① 必须先捕获子类
    System.err.println("文件不存在: " + e.getMessage());
} catch (IOException e) {              // ② 再捕获父类
    System.err.println("读取失败: " + e.getMessage());
}
```

常见 I/O 异常

异常	场景	是否受检 (checked)
FileNotFoundException	文件不存在	✓ 必须 try-catch
IOException	通用 I/O 错误	✓ 必须 try-catch
EOFException	到达流末尾	✓
UnsupportedEncodingException	编码不支持	✓
SecurityException	权限不足	✗

受检异常必须处理——要么 try-catch，要么在方法签名里 throws。

字符编码陷阱

✗ 默认编码的坑

```
// Mac/Linux 默认 UTF-8
// Windows 默认 GBK/GB2312
new FileReader("中文.txt");
```

Windows 写入的 GBK 文件在 Mac 用默认 UTF-8 读 → 乱码!

✓ 明确指定 UTF-8

```
import java.nio.charset.StandardCharsets;

new InputStreamReader(
    new FileInputStream("data.txt"),
    StandardCharsets.UTF_8);
```

Java 11+ 最简洁

```
// 默认 UTF-8
String content = Files.readString(
    Paths.get("data.txt"));
```

最佳实践

版本	做法
Java 8	显式传 UTF_8
Java 11+	<code>Files.readString</code>
任何版本	禁止依赖默认编码

铁律：跨平台代码必须指定编码。

I/O 最小知识清单

刚入门记这三组就够：

```
// — 1. 读文件（按行） —————
try (BufferedReader br = new BufferedReader(new FileReader("in.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        process(line);
    }
}

// — 2. 写文件（按行） —————
try (BufferedWriter bw = new BufferedWriter(new FileWriter("out.txt"))) {
    bw.write("hello");
    bw.newLine();
}

// — 3. 控制台交互 —————
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
System.out.printf("n = %d\n", n);
```

学习顺序：Scanner → BufferedReader/Writer → try-with-resources → Files → 字符编码。

PART 2 · 大作业讲解

Project 05 Env 答案解析 · Project 06 Control 布置

大作业进度回顾

解释器流水线

用户输入字符串

- ↓ 【01-REPL】 交互式读取 ← 已完成 ✓
- ↓ 【02-Lexer】 词法分析 ← 已完成 ✓
- ↓ 【03-AST】 语法分析 ← 已完成 ✓
- ↓ 【04-Eval】 求值 ← 已完成 ✓
- ↓ 【05-Env】 环境与绑定 ← 答案解析 ★
- ↓ 【06-Control】 控制流 ← 本次布置 ★

↓

输出结果

Project 05 答案 — 整体变化

Project 04 → 05 的关键演变:

方面	Project 04	Project 05
符号查找	<code>globals.get()</code>	<code>env.lookup()</code> 沿 parent 链
eval 签名	<code>eval(obj)</code>	<code>eval(obj, env)</code>
特殊形式	无	<code>let</code> / <code>let*</code> / <code>if</code> / <code>begin</code>
作用域	只有全局	嵌套作用域链

新增 4 个 SYMBOL 常量

```
private static final LSymbol SYMBOL_LET = new LSymbol("let");
private static final LSymbol SYMBOL_LET_STAR = new LSymbol("let*");
private static final LSymbol SYMBOL_IF = new LSymbol("if");
private static final LSymbol SYMBOL_BEGIN = new LSymbol("begin");
```

为什么用常量? 避免每次分发时 `new LSymbol("let")` ——性能优化 + 避免拼写错误。

Project 05 答案 — Environment 类

```
public class Environment {
    private final Environment parent;                // 父环境引用
    private final HashMap<LSymbol, LObject> symbols = new HashMap<>();

    public Environment() { this.parent = null; }      // 全局环境
    public Environment(Environment parent) { this.parent = parent; } // 子环境

    public void define(LSymbol symbol, LObject value) {
        symbols.put(symbol, value);
    }

    public LObject lookup(LSymbol symbol) {
        if (symbols.containsKey(symbol)) return symbols.get(symbol); // ① 当前层查找
        if (parent != null) return parent.lookup(symbol);           // ② 递归查父
        throw new IllegalArgumentException("Invalid symbol: " + symbol); // ③ 未找到
    }
}
```

链式查找 = 作用域链：子环境找不到就问父亲，父亲找不到就问祖父，一直到 GLOBALS。

Project 05 答案 — handleLet 核心代码

```
private static LObject handleLet(LList list, Environment env, boolean inplace) {
    LList bindings = (LList) list.content.get(1);           // 绑定列表 ((x 1) (y 2))
    Environment newEnv = new Environment(env);               // 创建子环境

    for (LObject bindingObj : bindings.content) {
        LList binding = (LList) bindingObj;
        newEnv.define(
            (LSymbol) binding.content.get(0),                // 变量名
            eval(binding.content.get(1),                      // 求值表达式
                inplace ? newEnv : env)                      // ★ 关键：求值环境
        );
    }
    return eval(list.content.get(2), newEnv);               // 在新环境中执行 body
}
```

inplace 参数是 let 和 let* 的唯一区别：

- **let** (inplace=false) → 绑定在**旧环境** env 求值 (parallel bindings)
- **let*** (inplace=true) → 绑定在**新环境** newEnv 求值 (sequential bindings)

Project 05 答案 — let vs let* 求值示例

```
(let ((x 2) (y 3)) (let ((x 7) (z (+ x y))) (* z x))) → 35
(let ((x 2) (y 3)) (let* ((x 7) (z (+ x y))) (* z x))) → 70
```

let 执行（内层）

绑定 x=7:
eval(7, env) = 7

绑定 z=(+ x y):
在旧 env 求值
x=2, y=3 → z=5

body (* z x):
z=5, x=7 → 35

let* 执行（内层）

绑定 x=7:
eval(7, newEnv) = 7

绑定 z=(+ x y):
在新 newEnv 求值
x=7（新）, y=3（继承）
→ z=10

body (* z x):
z=10, x=7 → 70

唯一区别： z 求值时用的是旧 env 还是新 newEnv。

Project 05 答案 — 扩展1 handleIf

```
private static LObject handleIf(LList list, Environment env) {
    LObject cond = eval(list.content.get(1), env);           // 求值条件

    boolean isFalse = (cond instanceof LList)
        && ((LList) cond).content.isEmpty();                // 只有空列表是"假"

    if (!isFalse) {
        return eval(list.content.get(2), env);               // then 分支
    } else {
        return eval(list.content.get(3), env);               // else 分支
    }
}
```

"假"的定义 (Project 05)

值	真假	示例
nil / ()	假	(if nil 10 20) → 20
0	真	(if 0 10 20) → 10
任意非空列表	真	(if (cons 1 nil) 10 20) → 10

注意：Project 05 中 0 是"真"! 这与 Project 06 不同 (06 中 0 也是假)。

Project 05 答案 — 扩展2 handleBegin

```
private static LObject handleBegin(LList list, Environment env) {  
    LObject result = new LList();           // 默认空  
    for (int i = 1; i < list.content.size(); i++) { // 从 i=1 开始跳过 "begin"  
        result = eval(list.content.get(i), env); // 依次求值  
    }  
    return result;                           // 返回最后一个  
}
```

执行示例

```
(begin (+ 1 2) (* 3 4))  
i=1: eval (+ 1 2) → 3 (丢弃)  
i=2: eval (* 3 4) → 12 (保留)  
返回 12
```

begin 是最简单的特殊形式——就是个循环，只保留最后一个结果。

Project 05 答案 — eval 核心分发

```
private static LObject eval(LObject obj, Environment env) {
    if (obj instanceof LSymbol) {
        return env.lookup((LSymbol) obj); // 符号查找
    } else if (obj instanceof LList) {
        LList list = (LList) obj;
        if (list.content.isEmpty()) return obj; // 空列表
        else if (SYMBOL_LET.equals(list.content.get(0)))
            return handleLet(list, env, false); // let
        else if (SYMBOL_LET_STAR.equals(list.content.get(0)))
            return handleLet(list, env, true); // let*
        else if (SYMBOL_IF.equals(list.content.get(0)))
            return handleIf(list, env); // if
        else if (SYMBOL_BEGIN.equals(list.content.get(0)))
            return handleBegin(list, env); // begin
        else {
            LList evaluated = new LList(list.content.stream()
                .map(o -> eval(o, env)).collect(Collectors.toList()));
            return ((LFunction) evaluated.content.get(0)).call(evaluated); // 函数调用
        }
    } else return obj; // 字面量
}
```

Project 05 答案 — 普通函数调用详解

两行关键代码

```
LList evaluated = new LList(  
    list.content.stream()  
        .map(o -> eval(o, env))  
        .collect(Collectors.toList()));  
return ((LFunction) evaluated  
    .content.get(0)).call(evaluated);
```

第 1 步：批量求值子元素

`stream().map().collect()` 等价于：

```
List<LObject> r = new ArrayList<>();  
for (LObject o : list.content) {  
    r.add(eval(o, env));  
}
```

对每个子元素递归调用 `eval`，把原始 AST 节点变成求值后的值。

第 2 步：函数调用

求值过程 (+ 1 2)

```
list.content:  
    [LSymbol(+), LInteger(1), LInteger(2)]  
    ↓ stream.map(eval)  
evaluated.content:  
    [LFunction(+), LInt(1), LInt(2)]  
    get(0)      get(1)  get(2)  
    ↓  
LFunction.call(evaluated)  
    → 1 + 2 = LInteger(3)
```

三个关键点：

- `get(0)` 是函数本身
- `get(1)` 起才是参数
- `call` 传整个列表

Project 05 答案 — 教学要点总结

五个关键点

1. 作用域链的设计

子环境通过 `parent` 指针连接, `lookup` 递归查找——与 JavaScript 闭包、Python 嵌套函数同源。

2. `let` 与 `let*` 的统一

用一个 `inplace` 布尔参数区分, 避免代码重复——优秀的抽象。

3. 特殊形式 vs 普通调用

`let` / `let*` / `if` / `begin` 不先求值所有参数, 直接传入原始 AST 让 handler 自行决定求值顺序。

五个关键点 (续)

4. 变量遮蔽 (Shadowing)

`(let ((x 1)) (let ((x 2)) x))` → 2。子环境的 `x` 遮蔽父环境的 `x`——与 Java 局部变量遮蔽成员变量同理。

5. `if` 懒求值的重要性

```
(if nil (/ 1 0) 42) → 42
```

若 `if` 先求值所有参数, `(/ 1 0)` 会除零错误! 特殊形式使这成为可能。

Lisp 的 "code is data": `if/let` 本质上是编译时的语法变换, 而非运行时的函数调用。

Project 06 — Control 控制流概览

引入三个核心控制构造 + 两个扩展构造：

Project 06 新增特殊形式	
do	– 顺序执行
if	– 条件分支 (0 也是"假"了)
fn*	– 函数闭包 (最重要!)
扩展1: cond – 多分支条件 (switch)	
扩展2: when – 无 else 的 if + do	

图灵完备：有了 `fn*` 和 `if`，我们的解释器就能表达任意可计算的函数——这是 Lambda 演算的核心。

Project 06 — 图灵完备是什么？

什么是图灵完备（Turing Complete）？

一个计算模型如果能**模拟任意图灵机**，就称其图灵完备。等价地：**它能计算任何可计算的函数**。

三个等价的计算模型

模型	提出者	年代
图灵机（Turing Machine）	Alan Turing	1936
Lambda 演算（ λ -calculus）	Alonzo Church	1936
μ -递归函数	Kurt Gödel	1934

Church–Turing 论题：这三个模型等价，任意"可计算"的函数都可被它们之一表达。后来发现它们都等价于：寄存器机、Post 系统、细胞自动机.....

Lambda 演算**只有三样东西：变量、函数抽象、函数应用**。就这么简单，却图灵完备。

Project 06 — 为什么 `fn*` + `if` 就够了？

① 变量绑定 → `fn*` 自带

```
((fn* (x) body) value)
≡ let x = value in body
```

函数应用即是"把实参代入形参"，这就是变量绑定。

② 条件分支 → `if` 提供

没有 `if`，程序只能"一条路走到底"，无法根据数据决定走向——这是"计算"的本质之一。

③ 循环/递归 → `fn*` + `if`

```
(def fact (fn* (n)
  (if (= n 0)
    1
    (* n (fact (- n 1))))))
```

递归 = 循环——任何 `while` 都能改写为递归。`if` 负责终止条件，`fn*` 负责自调用。

④ 数据结构 → 纯函数编码

Lambda 甚至可以用纯函数编码整数、布尔值、pair，无需任何原生类型（下一页 Church 编码示例）。

5 个能力 = 通用编程语言的最小内核——变量、分支、递归、数据、顺序。

Project 06 — Church 编码示例

用纯 Lambda 表达布尔值

```
TRUE  = (fn* (x y) x)      ; 取第一个
FALSE = (fn* (x y) y)      ; 取第二个

(IF TRUE "yes" "no") → "yes"
(IF FALSE "yes" "no") → "no"
```

Church 数字

```
ZERO  = (fn* (f x) x)          ; f^0
ONE   = (fn* (f x) (f x))      ; f^1
TWO   = (fn* (f x) (f (f x)))  ; f^2
THREE = (fn* (f x) (f (f (f x))))
```

整数 N = "把 f 应用 N 次"。

加法的定义

```
PLUS = (fn* (m n)
  (fn* (f x) (m f (n f x))))
```

(PLUS TWO ONE) : TWO 应用 f 两次，初值是 (ONE f x)
= (f x)，结果 f(f(f(x))) = THREE

为什么重要？

启示	解释
纯函数足够	无需整数 / 数组
数据 = 行为	数字本质是迭代
语言极简	3 构造图灵完备

我们的 Lambda 解释器 == 完整通用计算机。

Project 06 — 图灵完备的实践意义

对大作业的启发

Project 06 完成后，你拥有：

能力	来源
变量绑定	<code>let</code> / <code>let*</code> / <code>fn*</code>
分支选择	<code>if</code> / <code>cond</code> / <code>when</code>
无限循环	<code>fn*</code> + 递归调用
数据组合	<code>cons</code> + <code>car</code> + <code>cdr</code>
顺序执行	<code>do</code>

这 5 个能力 = 通用编程语言的最小内核

历史意义

- **1936**：Church 和 Turing 独立证明这两个模型等价
- **1958**：McCarthy 发明 Lisp——第一个基于 Lambda 演算的实用语言
- **1970s**：Scheme 将 Lisp 简化为"纯" Lambda 演算 + 少量原语
- **今天**：JavaScript、Python、Java Lambda、Rust 闭包.....都继承自 Lambda 演算

反向启示

很多"高级特性"其实是**语法糖**：

```
for (i=0; i<n; i++) {...}
  ↓ 本质
(fn* (i) (if (< i n) ... (recur (+ i 1))))
```

学会了 Lambda 演算，你就看穿了所有编程语言的本质——它们只是在图灵完备的基础上加了不同的糖衣。

Project 06 — "假"值新约定

Project 05 → 06 的重要变化：

值	Project 05	Project 06
<code>nil</code> / <code>()</code>	假	假
<code>0</code>	真	假 ← 变化!
字符串	真	真
非空列表	真	真

判断代码

```
boolean isFalse = cond.equals(new LList())           // 空列表为假
                  || cond.equals(new LInteger(0));      // 0 也为假
```

更接近 C / Java 的习惯——0 为假，非 0 为真。

Project 06 — 任务1: handleDo 顺序执行

语法

```
(do expr1 expr2 ... exprN)
```

依次求值所有表达式，返回**最后一个**的结果。

示例

```
(do (* 1 2) (- 1 2) (+ 1 2)) → 3  
(do 42)                        → 42
```

提示代码

```
private static LObject handleDo(LList list, Environment env) {  
    // 从 i=1 到 size-2 执行 (结果丢弃)  
    // 返回 eval(list.content.get(size-1), env)  
    return null;  
}
```

注意：这和 Project 05 的 begin 几乎一样——只是名字改了。

Project 06 — 任务2: handleIF 条件求值

语法与规则

(if cond then else)

1. 求值 `cond`
2. 不是 `nil` 且不是 `0` → 求值 `then`
3. 是 `nil` 或 `0` → 求值 `else`

示例

```
(if 0 1 (+ 1 -1)) → 0    (0 是假)
(if 1 5 10)        → 5
(if nil 10 20)      → 20
```

提示代码

```
private static LObject handleIF(
    LList list, Environment env) {
    LObject cond = eval(
        list.content.get(1), env);

    boolean isFalse =
        cond.equals(new LList())
        || cond.equals(new LInteger(0));

    if (!isFalse)
        return eval(list.content.get(2), env);
    else
        return eval(list.content.get(3), env);
}
```

注意：和 Project 05 的 if 唯一区别是 0 也算假。

Project 06 — 任务3: handleFnStar 函数闭包

语法

```
(fn* (params...) body)
```

创建一个函数对象，**捕获当前环境**作为闭包。

示例

```
((fn* (a) a) 123)           → 123    (恒等函数)
((fn* (a b) (+ a b)) 3 4)   → 7      (两参数加法)

(let* ((x 10)
      (f (fn* (y) (+ x y))))
  (f 5))                    → 15      (闭包捕获 x)
```

什么是闭包？ 函数在创建时"记住"它所在的环境变量。即使后续环境变化，闭包内部依然能访问创建时的变量值。

Project 06 — handleFnStar 实现提示

```
private static LFunction handleFnStar(LList list, Environment env) {  
    LList params = (LList) list.content.get(1);           // 参数列表 (a b c)  
    LObject body = list.content.get(2);                   // 函数体  
  
    Environment captured = env.deepClone();               // ★ 关键：深拷贝！  
  
    return new LFunction(params.content.size(),  
        args -> eval(body, new Environment(captured, params, args)));  
}
```

为什么要 deepClone?

```
(let* ((x 10)  
      (f (fn* (y) (+ x y)))    ← f 捕获了 {x=10}  
      (x 99))                 ← 之后 x 被改为 99  
  (f 5))
```

如果不 deepClone, `f` 引用的是**同一个** let* 环境, 后续 `x=99` 会"泄露"进闭包 → 错误!

deepClone 创建环境快照, 确保闭包永远看到创建时刻的变量值。

Project 06 — Environment 新特性

```
// 新增的三参数构造器 (已提供)
public Environment(Environment parent, LList binds, LList args) {
    this.parent = parent;
    for (int i = 0; i < binds.content.size(); i++) {
        LSymbol symbol = (LSymbol) binds.content.get(i);
        symbols.put(symbol, args.content.get(i + 1));    // +1 因为 get(0) 是函数自身
    }
}

// 新增的 deepClone 方法 (已提供)
public Environment deepClone() {
    Environment clonedParent =
        (this.parent != null) ? this.parent.deepClone() : null;
    Environment clonedEnvironment = new Environment(clonedParent);
    for (HashMap.Entry<LSymbol, LObject> entry : this.symbols.entrySet()) {
        clonedEnvironment.symbols.put(entry.getKey(), entry.getValue());
    }
    return clonedEnvironment;
}
```

这两个方法 已经提供, 你只需要在 `handleFnStar` 中正确使用它们。

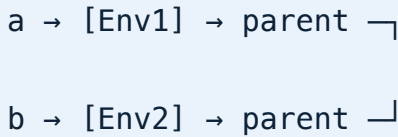
Project 06 — 浅拷贝 vs 深拷贝（概念）

三种"拷贝"对比

方式	做什么	特点
引用赋值 <code>b = a</code>	两变量指向同一对象	完全不是拷贝
浅拷贝 <code>new Env(a)</code>	复制顶层对象，内部引用共享	快，但不独立
深拷贝 <code>a.deepClone()</code>	递归复制所有层级	慢，完全独立

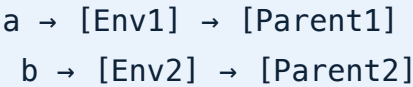
内存模型对比

浅拷贝：



⚠️ parent 共享！修改父环境会影响两边

深拷贝：



✅ 完全独立

本质区别：浅拷贝只复制"引用值"（指针），深拷贝递归复制"被指向的对象"。

Project 06 — 浅拷贝 vs 深拷贝（代码）

浅拷贝的代码

```
public Environment(Environment other) {  
    this.parent = other.parent;  
    // ⚠ parent 是共享引用  
  
    this.symbols = new HashMap<>(  
        other.symbols);  
    // symbols 本身新的，但值对象共享  
}
```

问题：

- `parent` 指针指向**同一个父环境**
- 修改父环境会影响副本
- 对于闭包场景完全不够用

深拷贝的实现

```
public Environment deepClone() {  
    // ① 递归深拷贝 parent  
    Environment p = (parent != null)  
        ? parent.deepClone() : null;  
  
    // ② 新建 Environment  
    Environment c = new Environment(p);  
  
    // ③ 复制每个键值对  
    for (Entry<LSymbol, LObject> e  
        : symbols.entrySet())  
        c.symbols.put(  
            e.getKey(), e.getValue());  
    return c;  
}
```

关键：第 ① 行的递归 `parent.deepClone()` 保证了整条 `parent` 链都被复制。

Project 06 — 为什么闭包需要深拷贝？

反例代码

```
(let* ((x 10)
      (f (fn* (y) (+ x y)))) ; 捕获 x=10
      (x 99))                ; 后续改为 99
(f 5))
```

正确答案应该是 15（闭包中的 x 永远是 10）。

浅拷贝执行过程 ❌

1. env0 = {x=10}
2. 创建 f, 浅拷贝 env0
captured 和 env0 共享符号表
3. 绑定 x=99 → env0 = {x=99}
captured 也变成 {x=99}!
4. (f 5) → lookup(x) → 99
返回 104 ❌

深拷贝执行过程 ✅

1. env0 = {x=10}
2. 创建 f, 深拷贝 env0
captured 独立副本 {x=10}
3. 绑定 x=99 → env0 = {x=99}
captured 仍是 {x=10} ✓
4. (f 5) → lookup(x) → 10
返回 15 ✅

函数式的核心承诺

闭包 = 函数 + 创建时的环境快照

这个"快照"必须不可变——否则闭包行为会随外部状态改变而改变，破坏引用透明性。

Project 06 — 扩展1: cond 多分支条件

语法与示例

```
(cond (test1 expr1)
      (test2 expr2)
      (else exprN))
```

```
(cond (1 10) (1 20) (else 30)) → 10
(cond (0 10) (1 20) (else 30)) → 20
(cond (0 10) (0 20) (else 30)) → 30
(cond (0 10) (0 20))           → ()
```

规则：依次求值 test，第一个"真"→执行对应 expr；`else` 符号无条件执行。

实现提示

```
private static LObject handleCond(
    LList list, Environment env) {
    for (int i=1; i<list.content.size(); i++) {
        LList clause = (LList) list.content.get(i);
        LObject test = clause.content.get(0);

        if (SYMBOL_ELSE.equals(test))
            return eval(clause.content.get(1), env);

        LObject r = eval(test, env);
        boolean isFalse = r.equals(new LList())
            || r.equals(new LInteger(0));
        if (!isFalse)
            return eval(clause.content.get(1), env);
    }
    return new LList(); // 无匹配
}
```

Project 06 — 扩展2: when 条件执行

语法

```
(when test expr1 expr2 ... exprN)
```

- `test` 为真 → 依次执行所有 `expr`, 返回最后一个
- `test` 为假 → 返回 `LList()`

示例

```
(when 1 (+ 1 2) (* 3 4)) → 12
```

```
(when 0 (+ 1 2) (* 3 4)) → ()
```

`when` ≈ `if` + `do` 组合 (没有 `else` 分支)。

实现提示

```
private static LObject handleWhen(  
    LList list, Environment env) {  
    LObject cond = eval(list.content.get(1), env);  
  
    boolean isFalse =  
        cond.equals(new LList())  
        || cond.equals(new LInteger(0));  
  
    if (isFalse) return new LList();  
  
    // 真: 依次执行 expr  
    LObject result = new LList();  
    for (int i = 2; i < list.content.size(); i++) {  
        result = eval(list.content.get(i), env);  
    }  
    return result;  
}
```

Project 06 — 任务总览与测试

6 个 TODO

TODO	内容	难度
1	<code>handleDo</code> 顺序执行	★
2	<code>handleIF</code> 条件求值	★
3	<code>handleFnStar</code> 函数闭包	★★★★
4	扩展1 <code>handleCond</code>	★★
5	扩展2 <code>handleWhen</code>	★
6	<code>eval</code> 核心分发	★★

难点：handleFnStar

- 必须 `deepClone` 环境
- 理解 `Environment(parent, binds, args)` 构造器的参数索引
- Lambda 捕获的 `captured` 变量

明测试（18 个）

分类	测试数
do 基础	2
if 三种情况	3
fn* 基础+闭包	3
综合嵌套	2
let/let* 回归	2
扩展1: cond	4
扩展2: when	2

运行测试

```
cd 06-control
mvn test
```

截止 下次课开始前

PART 3 · AI 编程进阶

上下文工程：System Prompt 工坊

上下文工程是什么？

上下文工程 (Context Engineering) = 系统化地设计、优化和管理输入给 LLM 的所有内容，使其产生可预期、可重复、高质量的输出

四个常见痛点

痛点	具体现象	根本原因
输出不稳定	相同问题，不同回答	没有约束输出空间
长对话失忆	第 10 轮忘记第 1 轮的约定	上下文窗口管理混乱
重复造轮子	每次都要重新解释项目背景	缺少可复用的 Skill
质量无法量化	不知道 Prompt 有没有变好	缺乏定量评估

Prompt 技巧是"术"，上下文工程是"道"——像管理代码一样管理 AI 行为。

什么是 System Prompt?

定义

System Prompt 是在对话开始之前注入给 LLM 的系统级约束，定义 AI 的：

- 角色身份
- 行为边界
- 输出格式
- 专业领域

类比：雇一个员工，第一天给他"员工手册"——System Prompt 就是这本手册。

有无 System Prompt 的对比

维度	无 System Prompt	有 System Prompt
输出风格	每次随机	固定、可预期
专业深度	通用泛化	聚焦领域
格式一致性	结构随意	固定模板
重复解释	每次都要	一次定义，永久生效

System Prompt 就像**函数签名的合约**——一旦定义，调用者（用户）就有了稳定的预期。

System Prompt 四要素框架

要素	作用	示例
① 角色 (Role)	定义 AI 的身份和专业领域	"你是资深 Java 后端工程师，精通 Spring Boot"
② 任务 (Task)	明确工作目标和职责边界	"负责代码审查，重点关注安全和性能"
③ 格式 (Format)	规定输出的结构和形式	"每个问题输出：【位置】【严重度】【改进方案】"
④ 背景 (Context)	注入项目的技术约束	"项目使用 Spring Boot 3.2 + PostgreSQL + Redis"

四要素缺一不可——缺少"格式"导致输出难以解析；缺少"背景"导致 AI 给出与项目不符的建议。

❌ 差的 vs ✅ 好的 System Prompt

❌ 差的示例

你是一个代码审查专家。
审查代码。

问题：

- 角色模糊（什么语言？什么经验？）
- 无输出格式约束
- 无技术栈背景
- 无严重度判断标准

结果： AI 乱评，假报警多，输出随机，无法集成到流程。

✅ 好的示例

你是资深 Java 后端代码审查专家
(5年+经验)。

【检查重点】

- 安全：SQL 注入 · XSS · 权限验证
- 性能：N+1 · 缓存 · 算法复杂度
- 可维护性：命名 · 文档 · 重复代码

【输出格式】

每个问题：

【位置】 文件:行号

【严重度】 ●高 / ●中 / ●低

【改进】 代码示例

【技术栈】

Spring Boot 3.2 · PostgreSQL 16 · Redis 7

【原则】

只报告 90% 确信的问题；不确定时说
"建议确认"而非直接报错

System Prompt 是"活文档"——持续迭代

版本	改进点	效果变化
v1.0	基础角色定义	输出随机，假报警 40%
v1.1	增加检查清单	假报警降至 25%
v1.2	加入格式规范	输出结构一致，可解析
v1.3	加入"高置信度原则"	假报警降至 10% 以下

迭代方法论

- 1. 记录失败案例 — 每次 AI 输出不符合预期时，记录下来
- 2. 找共同模式 — 3 个以上类似失败 → 修改 System Prompt
- 3. 版本控制 — 用 Git 管理 System Prompt，追踪改进历史

把 System Prompt 当代码来维护——它就是你的 AI 助手的"源代码"。

System Prompt 的安全边界 — 什么不该放

类别	危险做法	风险
API 密钥 / 凭证	在 Prompt 中写 <code>OPENAI_KEY=sk-xxx</code>	日志、缓存、第三方服务均可能泄露
用户 PII	拼入真实姓名、身份证号、手机号	违反 GDPR / 个人信息保护法
公司机密源代码	把核心算法整段发给公共 API	商业机密上传到第三方服务器
生产数据库内容	把真实用户订单作为上下文	数据合规风险，泄露不可撤回

企业场景下的合规原则

使用公共 AI API 时，默认你发送的所有内容都可能被：

- ① 用于模型训练
- ② 在数据泄露时被访问
- ③ 被运营商员工审查

建议：涉及机密数据时使用**私有化部署**（本地 Ollama / 企业版 API）或严格脱敏后发送。

动手实验 — 为大作业设计 System Prompt

任务：用四要素框架为你的 Lambda 演算解释器设计一个 System Prompt

要素	你的填写
① 角色	例：资深 Java 工程师 + 函数式编程专家
② 任务	例：帮我实现 06-Control 的 handleFnStar，解释闭包原理
③ 格式	例：1)思路分析 → 2)代码实现 → 3)边界情况 → 4)测试用例
④ 背景	例：项目基于 edu.nju.software.*，LList 不可变，必须用 deepClone

完成后立即试用

粘贴到 Claude / ChatGPT 的 Custom Instructions（自定义指令），你的 AI 助手立刻变成你的项目专属助手。

如何设置 System Prompt (1/2) — 网页版

Claude (Anthropic)

Claude.ai 网页版

1. 左下角头像 → **Settings**
2. 选 **Profile** 标签
3. 在 "personal preferences" 框填写

推荐: Projects (项目级)

- 创建 Project → Project Settings
- 填 **Custom Instructions**
- 适合按项目隔离上下文

Claude Code (CLI)

- 项目根目录 `CLAUDE.md` (项目级)
- `~/.claude/CLAUDE.md` (全局)
- 内容自动作为 System Prompt

ChatGPT (OpenAI)

ChatGPT 网页版

1. 头像 → **Personalization**
2. 选 **Custom instructions**
3. 两个输入框:
 - "About me" (背景)
 - "How should ChatGPT respond" (行为)

自定义 GPT

- Explore → **Create a GPT**
- 填 Instructions、知识库、行动

Gemini (Google)

- 打开 `gemini.google.com`
- Settings → **Gems** (宝石)
- 创建 Gem 并填 Instructions

如何设置 System Prompt (2/2) — IDE 与 API

IDE 集成配置文件

工具	配置文件位置
Cursor	<code>.cursorrules</code> (项目根目录)
GitHub Copilot	<code>.github/copilot-instructions.md</code>
通义灵码	设置 → 编程偏好 → 自定义规则
Windsurf	<code>.windsurfrules</code> (项目根)

配置示例 (`.cursorrules`)

你是资深 Java 工程师。

- 遵循阿里巴巴 Java 规范
- 使用 Spring Boot 3.x
- 所有方法必须有 Javadoc
- 测试框架: JUnit 5 + Mockito

IDE 配置文件会自动注入到每次 AI 对话中。

API 方式 (编程调用)

OpenAI SDK

```
messages = [  
  {"role": "system",  
   "content": "你是 Java 专家..."},  
  {"role": "user",  
   "content": "写个 quicksort"}  
]
```

Anthropic SDK

```
client.messages.create(  
  system="你是 Java 专家...",  
  messages=[  
    {"role": "user",  
     "content": "写个 quicksort"}  
  ]  
)
```

关键差异: Claude API 的 `system` 是独立参数 OpenAI

当堂练习

当堂练习 · 独立完成 · 25 分钟

练习 1 — 按行读取 + 词频统计

15 min

```
import java.io.*;
import java.util.*;

public class WordCount {
    // 读取文件所有行，统计词频
    // 按空格分隔，不区分大小写
    static HashMap<String, Integer>
        countWords(String path)
        throws IOException {
        // 你的代码
    }
}
```

输入 `poem.txt`

```
Hello World
Hello Java
World is wonderful
```

期望输出

```
{hello=2, world=2, java=1,
 is=1, wonderful=1}
```

提示

1. try-with-resources + `BufferedReader`
2. `readLine()` 循环读到 null
3. `split("\\s+")` 按空白分词
4. `toLowerCase()` 统一小写
5. `getOrDefault` 累加计数

练习 2 — 文件复制与追加写入

10 min

```
public class FileCopier {  
    // 复制 src 到 dst, 每行加行号  
    static void copyWithLineNumber(  
        String src, String dst)  
        throws IOException { ... }  
  
    // 向 dst 追加一条日志  
    static void appendLog(  
        String dst, String msg)  
        throws IOException { ... }  
}
```

提示

- 复制: `BufferedReader` + `BufferedWriter`
- 追加: `new FileWriter(dst, true)`
- 行号计数器从 1 开始

期望效果

输入 `src.txt`: `hello\nworld`

调用 `copyWithLineNumber` 后:

```
1: hello  
2: world
```

再 `appendLog("dst.txt", "done")`:

```
1: hello  
2: world  
done
```

关键: 追加必须用 `new FileWriter(dst, true)`, 第二个参数 `true`。

课后作业

必做

1. 完成 Project 06 Control

提交到 SEECODER-Coder，截止下次课前。

六个 TODO: `do` · `if` · `fn*` · `cond` · `when` · `eval`

2. AI 辅助实验报告（一段话）

用 System Prompt 四要素框架为大作业定制一个 AI 助手，记录你的 Prompt 版本迭代过程（至少 2 个版本）

选做

- 用 `Files.lines()` 流式处理一个大文本文件，统计包含 `ERROR` 的行数
- 思考：为什么 `handleFnStar` 必须 `deepClone` 而不能直接引用环境？写个反例
- 对比 Project 05 的 `if` 和 Project 06 的 `if`，它们的“假”值定义不同，哪种设计更合理？

预习

下周内容：继承与多态（OOP 核心）

带着问题来：LFunction 和 LFunctionTco
如何通过继承/多态统一？

第七次编程实践课

软件工程与计算 I · 2026 · 南京大学

| 感谢聆听 · 有问题随时提问